¡Esta calculadora no funciona!

Noelia Velardo Rivas

1º DAM

Entornos de desarrollo

Parte 1: Diseño de pruebas

Tarea 1: Identificación de pruebas

Tarea 2: Definición de casos de prueba

Para probar la clase Calculadora, lo más importante son dos tipos de pruebas: las pruebas unitarias y las pruebas de caja negra.

Las pruebas unitarias son como probar cada parte de la calculadora por separado, para asegurarnos de que cada operación (sumar, restar, multiplicar, dividir) hace bien su trabajo. Por ejemplo, pruebas si la suma de 2 y 3 realmente da 5, o si la resta de 10 y 5 da 5. Así detectamos si alguna función tiene un error en su lógica.

Las pruebas de caja negra son como probar la calculadora sin mirar cómo está por dentro, solo comprobamos que, al darle ciertos números, saque el resultado que esperamos. También sirve para probar casos especiales, por ejemplo, qué pasa si alguien intenta dividir por cero, que es un caso error que puede hacer que la calculadora "se rompa".

Por eso, los dos tipos de pruebas son importantes para que estemos seguras que la calculadora funciona bien y también que gestione los errores sin fallar.

Caso de prueba	Entradas (a, b)	Resultado esperado
CP-DIV-01	(10, 2)	5
CP-DIV-02	(5, 2)	2.5 (o 2 si es división entera)
CP-DIV-03	(10, 0)	Error (ArithmeticException)
CP-DIV-04	(0, 5)	0.0 (límite)

CP-DIV-01 (10, 2): Situación normal. Es una división exacta entre dos números positivos.

CP-DIV-02 (5, 2): Sigue siendo un caso correcto, pero el resultado tiene decimales. Sirve para comprobar divisiones que no son exactas.

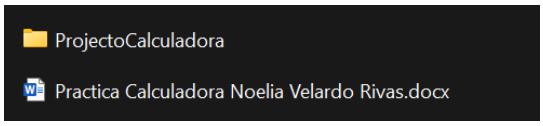
CP-DIV-03 (10, 0): Caso de fallo. Al dividir entre cero debe producirse una ArithmeticException, así se revisa cómo se gestionan los errores.

CP-DIV-04 (0, 5): Caso de frontera. El numerador vale cero y el divisor es positivo, por lo que el resultado esperado es 0.

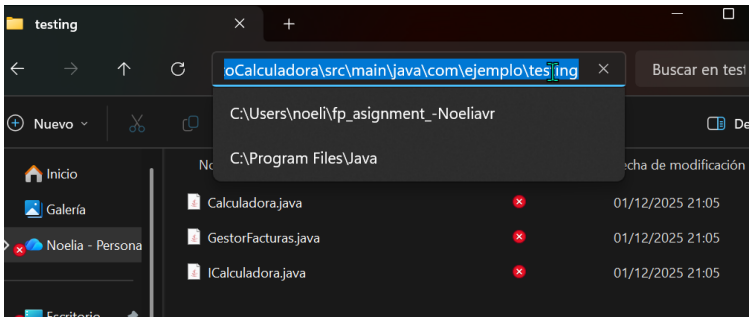
Parte 2: Pruebas automatizadas con Junit

Tarea 3: Implementación de pruebas

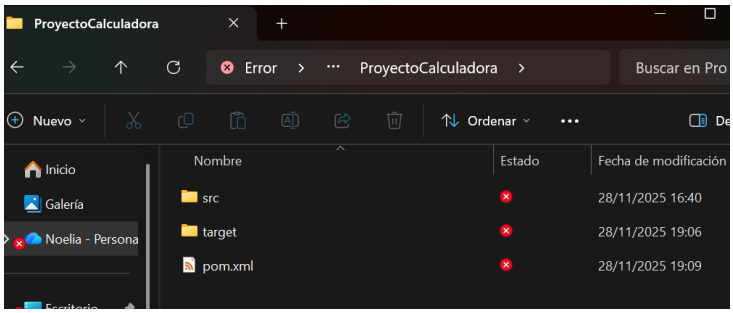
Paso 1: Crear un archivo en donde ira calculadoratest.java, calculadora.java y el pom.xml necesario para usar JUnit con Maven.

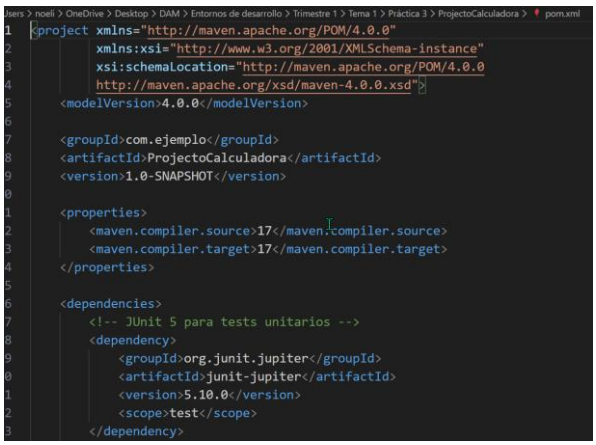


Paso 2: Estructura de las carpetas



Paso 3: Y el archivo pom.xml y en contenido que debe de llevar





```xml
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.ejemplo</groupId>
    <artifactId>ProyectoCalculadora</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
    </properties>

    <dependencies>
        <!-- JUnit 5 para tests unitarios -->
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>5.10.0</version>
            <scope>test</scope>
        </dependency>
```

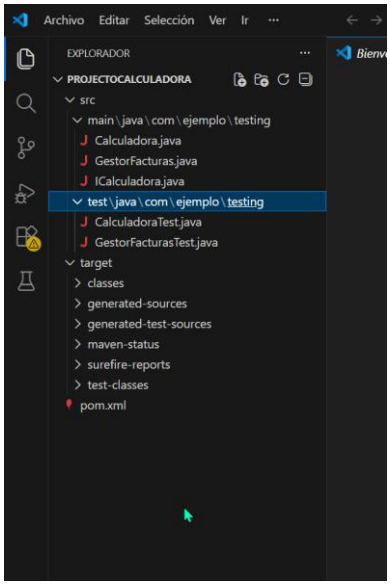
```
        <!-- Mockito Core para mocks -->
        <dependency>
            <groupId>org.mockito</groupId>
            <artifactId>mockito-core</artifactId>
            <version>5.5.0</version>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <!-- Plugin para ejecutar tests con Maven -->
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-surefire-plugin</artifactId>
                <version>3.1.2</version>
                <configuration>
                    <includes>
                        <include>**/*Test.java</include>
                    </includes>
                </configuration>
```

```
        </dependency>
    </dependencies>

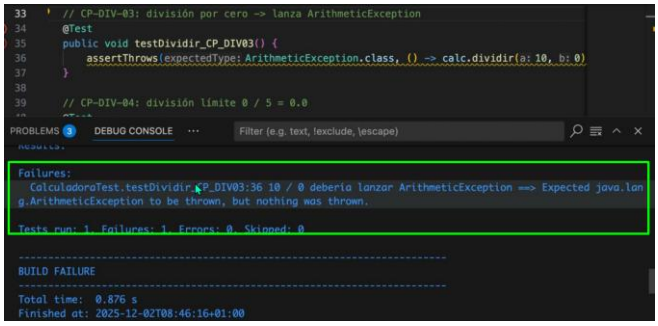
    <build>
        <plugins>
            <!-- Plugin para ejecutar tests con Maven -->
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-surefire-plugin</artifactId>
                <version>3.1.2</version>
                <configuration>
                    <includes>
                        <include>**/*Test.java</include>
                    </includes>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```

Paso 4: En caso de haberlo hecho todo correctamente debería de salir igual que en la imagen.

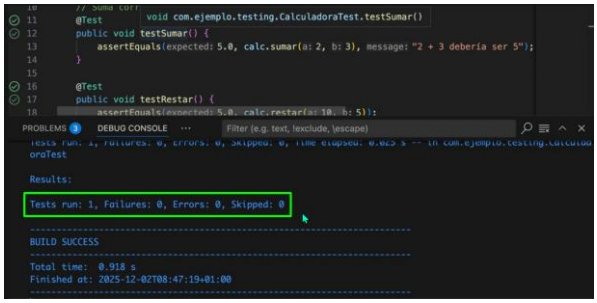


Paso 5: Hacer la prueba de que no de ningún error

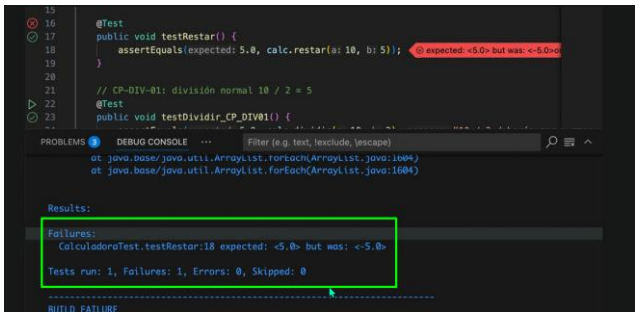
La división entre 0, da error



Prueba testSumar



Prueba resta, da error porque da como resultado -5, en vez de dar 5



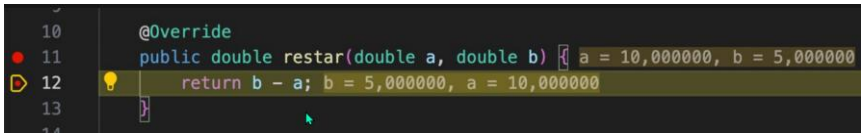
Parte 3: Depuración e investigación del error

Tarea 4: Identificar y usar herramientas

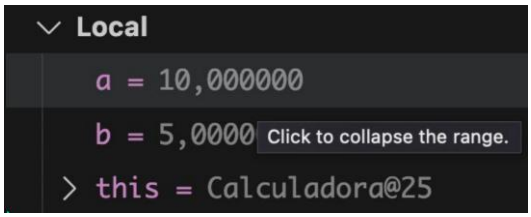
Tarea 5: Examinar y modificar en ejecución

Paso 1: error y corrección de la resta

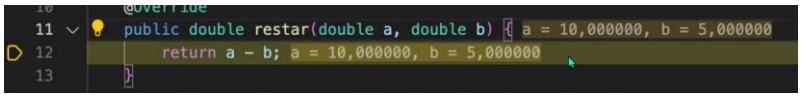
Error de la resta



Paso 2: Para cambiar los valores hacerlo donde aparece en la imagen



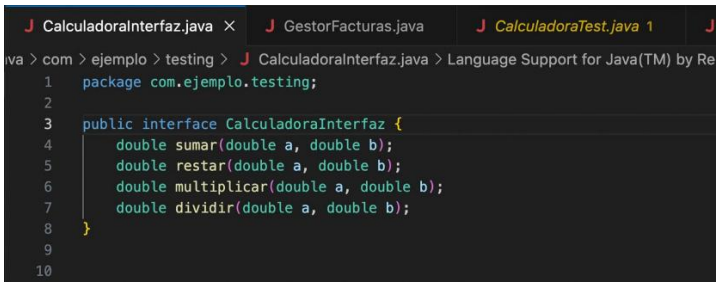
Paso 3: Corrección de la resta



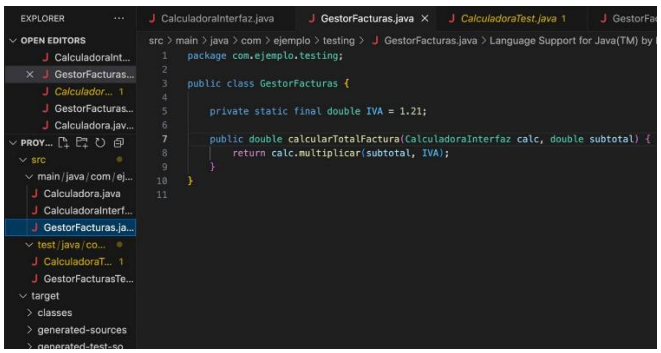
Parte 4: Aislamiento con dobles pruebas

Tarea 6: Uso de dobles de pruebas

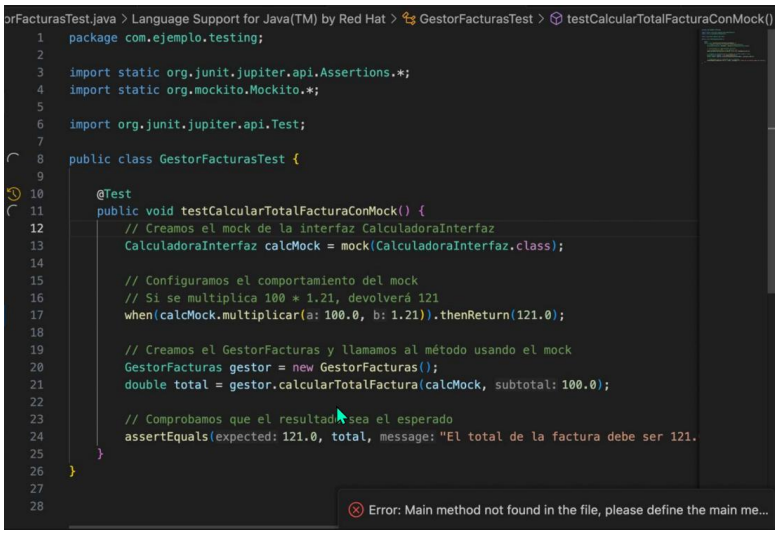
Paso 1: Creamos el archivo .java para evitar problemas con clases como se ve en la imagen



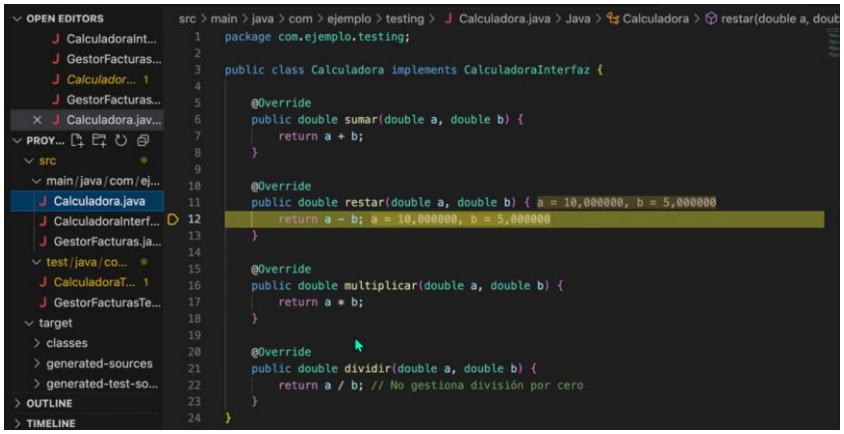
Paso 2: El archivo GestorFacturas quedaría como se ve en la imagen.



Paso 3: El archivo GestorFacturasTest quedaría como se ve en la imagen.



Paso 4: El archivo Calculadora quedaría como se ve en la imagen.



Parte 5: Documentación de incidencias

Tarea 7: Documentar la incidencia

- Título: “La aplicación crashea con AritmeticException al intentar dividir por cero”
- Prioridad: Alta/crítica
- Descripción: El método dividir (double a, double b) de la clase Calculadora realiza la operación a / b sin comprobar si el divisor b es igual que 0.
  Cuando se llama a este método con b=0; la Java Virtual Machine lanza una java.lang.ArithmeticException y la aplicación se detiene de forma inesperada.
- Pasos para reproducirlo:
  1. Crear una instancia de la clase Calculadora
  2. Llamar al método dividir (10,0) u otro valor cualquiera con b = 0.
  3. Ejecutar la prueba unitaria correspondiente.
- Comportamiento observado: Durante la ejecución, el programa lanza una excepción no controlada java.lang.ArithmeticException: / by zero y la prueba termina en error; la aplicación se detiene en lugar de gestionar la situación.
- Comportamiento esperado: El método dividir debería lanzar una excepción controlada o devolver un valor especial, de la forma que la aplicación no crashee y el error pueda gestionarse adecuadamente por la capa que llama al método.